PRACTICA 4: GESTION DE IMAGENES Y CONTENEDORES

POR:

ANDRES SANTIAGO BERMUDEZ MUÑOZ

SARA ISABEL ALEGRIA NARVAEZ

DOCENTE:

LEIDY MARIANI DORADO FLOREZ

SISTEMAS DISTRIBUIDOS

INGENIERÍA DE SISTEMAS

CORPORACION UNIVERSITARIA COMFACAUCA – UNICOMFACAUCA

POPAYÁN – CAUCA

2026

PARTE 1 – Exploración de Imágenes

Actividad 1: Ver imágenes locales

Ejecutamos:

- docker images

```
PS C:\Users\245G8-R5\OneDrive\Documentos\APIREST> docker images

IMAGE                  ID            DISK USAGE   CONTENT SIZE   EXTRA
demo-api-rest:latest   d12e448055cf       247MB           58MB    U
misitioweb:latest      e56664583f55       237MB         62.9MB    U
mysql:latest           e5dc14f6e01c      1.27GB          283MB    U
nginx:latest           0236ee02dcbc       240MB         65.8MB    U
sitio_web:latest       10800081c226       237MB         62.9MB    U
PS C:\Users\245G8-R5\OneDrive\Documentos\APIREST>
```

Respondan en equipo:

- ¿Qué es el IMAGE ID?
  Rta: El sistema utiliza un ID único para darle una identificación a cada imagen que se almacene. Este identificador sirve para diferenciar una imagen de otra, aunque tenga el mismo nombre o versión.

- ¿Qué significa TAG?
  Rta: Es una etiqueta que se utiliza para identificar y sacar una o varias versiones de una misma imagen Docker. Por ejemplo en el caso de mejorar el sistema permite tener la versión más antigua y la nueva se utilizará para la mejora, teniendo un respaldo por si necesita una versión más antigua.

- ¿Qué representa SIZE?
  Rta: El SIZE indica cuánto espacio de almacenamiento utiliza una imagen en el sistema, osea que muestra el tamaño que ocupa una imagen en el disco.

- ¿Una imagen se está ejecutando?
  Rta: La imagen no se ejecuta sola, debido a que solo es una plantilla o paquete que contiene la aplicación. Para que funcione debe crearse un contenedor que se utiliza para ejecutar la imagen.

Actividad 2: Descargar nuevas imágenes

Ejecutamos:

- docker pull redis

```
PS C:\Users\245G8-R5\OneDrive\Documentos\APIREST> docker pull redis
Using default tag: latest
latest: Pulling from library/redis
4f4fb700ef54: Pull complete
214ecf6e5c87: Pull complete
89b1b12d7109: Pull complete
141e1fc22067: Pull complete
ae77945b0371: Pull complete
f25f5c85b90d: Pull complete
e798a26a6510: Download complete
4a4db1a35d5b: Download complete
Digest: sha256:1c054d54ecd1597bba52f4304bca5afbc5565ebe614c5b3d7dc5b7f8a0cd768d
Status: Downloaded newer image for redis:latest
docker.io/library/redis:latest
PS C:\Users\245G8-R5\OneDrive\Documentos\APIREST>
```

Luego, ejecutamos nuevamente:

- docker images

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
demo-api-rest:latest	d12e448055cf	247MB	58MB	U
misitioweb:latest	e56664583f55	237MB	62.9MB	U
mysql:latest	e5dc14f6e01c	1.27GB	283MB	U
nginx:latest	0236ee02dcbc	240MB	65.8MB	U
redis:latest	1c054d54ecd1	204MB	55.3MB	
sitio_web:latest	10800081c226	237MB	62.9MB	U

Reflexionar:

- ¿Qué cambió?
  Rta: Al ejecutar el comando docker pull redis, se descargó la imagen de Redis que se agregó a la lista de imágenes locales y al ejecutar nuevamente docker imagen se puede observar la imagen redis:latest, lo que muestra que se descargó e instaló correctamente.

- ¿El servicio ya está funcionando?
  Rta: El servicio de redis no está ejecutado solo se instaló, para que se ejecute se necesita crear un contenedor a partir de esa imagen que se utilice para la instancia de ejecución.

PARTE 2 – Gestión de Contenedores

Actividad 3: Crear un contenedor

Ejemplo con nginx:

- docker run -d --name servidor-web-p8083:80 nginx

```
PS C:\Users\245G8-R5\OneDrive\Documentos\APIREST> docker run -d --name servidor-web -p 8083:80 nginx
37694c363e98ddce104194c835c5b0ddda89406c3b663948989cdeb616e1ac70
PS C:\Users\245G8-R5\OneDrive\Documentos\APIREST>
```

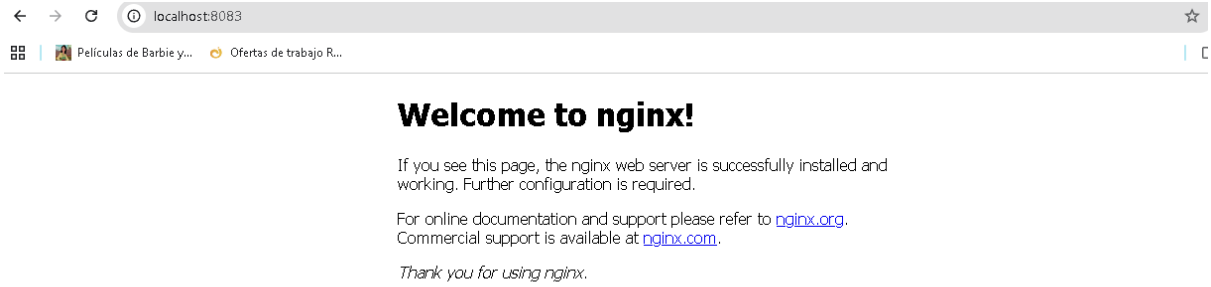
Verificar:

- docker ps

```
PS C:\Users\245G8-R5\OneDrive\Documentos\APIREST> docker ps
CONTAINER ID   IMAGE               COMMAND                  CREATED         STATUS         PORTS                                          NAMES
37694c363e98   nginx               "/docker-entrypoint.…"   40 seconds ago  Up 42 seconds  0.0.0.0:8083->80/tcp, [::]:8083->80/tcp        servidor-web
7eda86243163   demo-api-rest       "docker-entrypoint.s…"   2 hours ago     Up 2 hours     0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp    vigorous_keller
bcd256c6f78b   sitio_web:latest    "/docker-entrypoint.…"   3 hours ago     Up 3 hours     0.0.0.0:8082->80/tcp, [::]:8082->80/tcp        web3
bf72fb63e36b   misitioweb          "/docker-entrypoint.…"   21 hours ago    Up 21 hours    0.0.0.0:8081->80/tcp, [::]:8081->80/tcp        web
cca24cf7eacf   misitioweb:latest   "/docker-entrypoint.…"   21 hours ago    Up 21 hours    80/tcp                                         confident_darwin
c37c01f3cc96   nginx               "/docker-entrypoint.…"   21 hours ago    Up 21 hours    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp        clase6-web-1
41b3df54ca36   mysql               "docker-entrypoint.s…"   21 hours ago    Up 21 hours    3306/tcp, 33060/tcp                            clase6-db-1
PS C:\Users\245G8-R5\OneDrive\Documentos\APIREST>
```

Abrir el navegador:

- localhost:8083



Actividad 4: Administrar el contenedor

Detener:

- docker stop servidor-web

```
PS C:\Users\245G8-R5\OneDrive\Documentos\APIREST> docker stop servidor-web
servidor-web
```

Ver todos los contenedores:

- docker ps -a

```
PS C:\Users\245G8-R5\OneDrive\Documentos\APIREST> docker ps -a
CONTAINER ID   IMAGE               COMMAND                  CREATED         STATUS                     PORTS                                          NAMES
37694c363e98   nginx               "/docker-entrypoint.…"   4 minutes ago   Exited (0) 38 seconds ago                                         servidor-web
7eda86243163   demo-api-rest       "docker-entrypoint.s…"   2 hours ago     Up 2 hours                 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp    vigorous_keller
bcd256c6f78b   sitio_web:latest    "/docker-entrypoint.…"   3 hours ago     Up 3 hours                 0.0.0.0:8082->80/tcp, [::]:8082->80/tcp        web3
bf72fb63e36b   misitioweb          "/docker-entrypoint.…"   21 hours ago    Up 21 hours                0.0.0.0:8081->80/tcp, [::]:8081->80/tcp        web
cca24cf7eacf   misitioweb:latest   "/docker-entrypoint.…"   21 hours ago    Up 21 hours                80/tcp                                         confident_darwin
c37c01f3cc96   nginx               "/docker-entrypoint.…"   21 hours ago    Up 21 hours                0.0.0.0:8080->80/tcp, [::]:8080->80/tcp        clase6-web-1
41b3df54ca36   mysql               "docker-entrypoint.s…"   21 hours ago    Up 21 hours                3306/tcp, 33060/tcp                            clase6-db-1
```

Reiniciar:

- docker start servidor-web

```
PS C:\Users\245G8-R5\OneDrive\Documentos\APIREST> docker start servidor-web
servidor-web
```

Eliminar:

- docker rm -f servidor-web

```
PS C:\Users\245G8-R5\OneDrive\Documentos\APIREST> docker rm -f servidor-web
servidor-web
PS C:\Users\245G8-R5\OneDrive\Documentos\APIREST> docker ps -a
CONTAINER ID   IMAGE              COMMAND                  CREATED        STATUS        PORTS
7eda86243163   demo-api-rest      "docker-entrypoint.s…"   2 hours ago    Up 2 hours    0.0.0.0:3000->3000/tcp,
bcd256c6f78b   sitio_web:latest   "/docker-entrypoint.…"   3 hours ago    Up 3 hours    0.0.0.0:8082->80/tcp, [
bf72fb63e36b   misitioweb         "/docker-entrypoint.…"   21 hours ago   Up 21 hours   0.0.0.0:8081->80/tcp, [
cca24cf7eacf   misitioweb:latest  "/docker-entrypoint.…"   21 hours ago   Up 21 hours   80/tcp
c37c01f3cc96   nginx              "/docker-entrypoint.…"   21 hours ago   Up 21 hours   0.0.0.0:8080->80/tcp, [
41b3df54ca36   mysql              "docker-entrypoint.s…"   21 hours ago   Up 21 hours   3306/tcp, 33060/tcp
```

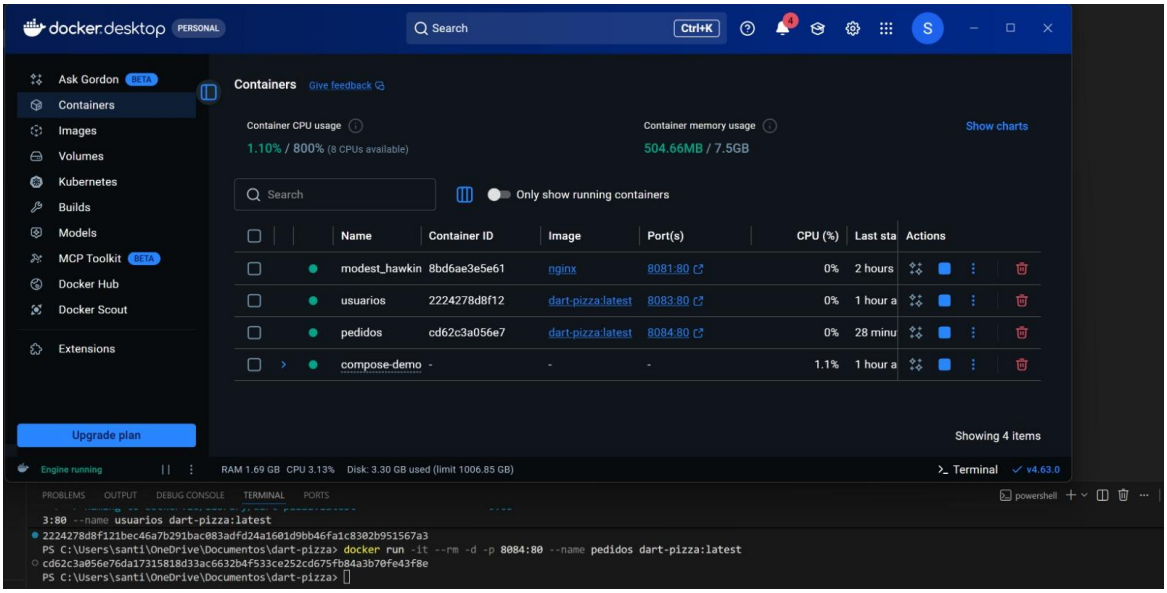
PARTE 3 – Análisis Conceptual

Completar esta tabla:

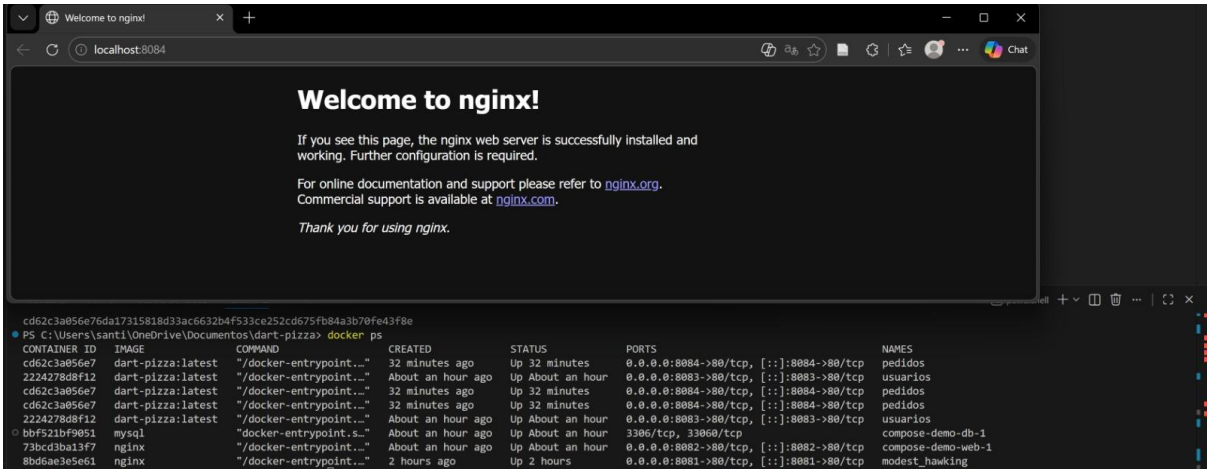
Concepto	Imagen	Contenedor
¿Es ejecutable?	No la imagen por sí sola no se ejecuta, solo sirve como una plantilla.	Si el contenedor se utiliza como la instancia para ejecutar una imagen
¿Ocupa recursos en ejecución?	No la imagen solo ocupa espacio de almacenamiento en el disco	Si al ejecutarse utiliza recursos de la CPU, memoria Ram y red.
¿Puede modificarse?	No para poder modificarse se debe crear una nueva imagen o una versión diferente	Si el contenedor permite modificarse
¿Puede eliminarse?	Si una imagen se puede eliminar del sistema, pero no debe estar siendo utilizado por un contenedor	Si el contenedor se puede eliminar cuando se desee

PARTE 4 – Reto en Equipo

1. Ejecutar un contenedor diferente.



2. Mostrar que está activo:



3. Explicar:
   a. ¿Qué tipo de servicio es?

      Es un servicio web, debido a que en nuestro caso el contenedor está utilizado NGINX para que funcione como servicios web y se encargue de atender solicitudes HTTP y mostrar páginas web a los usuarios a través del navegador

   b. ¿En qué tipo de sistema distribuido podría usarse?

      Se puede utilizar en arquitectura de microservicios o aplicaciones web, donde se necesite de varios servicios en diferentes contenedores, pero que al final trabajen juntos para el correcto funcionamiento de un sistema.

c. ¿Qué pasaría si ese servicio falla?

   Si el servicio falla los usuarios no podrían acceder a la página web o servicios que ofrece ese servidor. Pero en caso de sistema distribuido, se busca utilizar otros servidores o contenedores de respaldo para mantener el servicio disponible en el caso de que alguno falle.

PARTE 5 – Análisis Crítico

Responder:

1. ¿Cuál es la diferencia real entre imagen y contenedor?

   Rta: la diferencia es que la imagen es una plantilla que contiene la aplicación para funcionar, esta no se ejecuta por sí sola, y el contenedor es una copia de la imagen que se ejecuta en ese momento, es donde esta se ejecuta y utiliza recursos del sistema como memoria y red.

2. ¿Por qué Docker facilita la escalabilidad?

   Rta: docker hace que sea más fácil aumentar o disminuir el número de servicios según sea necesario, ya que crea muchos contenedores de una misma imagen de forma rápida, esto permite distribuir la carga entre varios de ellos y mejorar el rendimiento de la aplicación.

3. ¿Cómo se relaciona esto con los microservicios?

   Rta: se relacionan porque cada microservicio puede ejecutarse, actualizarse, escalar o reiniciarse de manera independiente en un contenedor distinto, sin afectar los demás servicios del sistema, lo que facilita el mantenimiento y creación de aplicaciones distribuidas.

4. ¿Qué ventajas ofrece frente a instalar software directamente?

   Rta: permite separar las aplicaciones y sus dependencias, esto evita conflictos entre distintos programas, facilita la portabilidad ya que una aplicación puede funcionar de la misma forma en distintos entornos y además facilita la instalación, el despliegue y la gestión de aplicaciones en sistemas distribuidos.

Conclusión grupal (mínimo 5 líneas)

Rta: docker es una herramienta valiosa para desarrollar e implementar aplicaciones modernas, ya que esto permite ejecutar servicios de forma individual a través de contenedores. A través de imágenes, se pueden crear muchos contenedores que funcionan

con aplicaciones de forma rápida y eficiente, lo que facilita escalar y mantener sistemas distribuidos. Además, docker asegura que las aplicaciones funcionen de igual forma en diferentes entornos sin problemas de dependencia. En conclusión, usar contenedores mejora la organización, portabilidad y eficiencia en el desarrollo de software.

Rta:
Docker es una herramienta muy útil para el desarrollo y despliegue de aplicaciones modernas, ya que permite ejecutar servicios de forma aislada mediante contenedores. A través de imágenes se pueden crear múltiples contenedores que ejecutan aplicaciones de manera rápida y eficiente. Esto facilita la escalabilidad y el mantenimiento de sistemas distribuidos. Además, Docker permite que las aplicaciones se ejecuten de forma consistente en diferentes entornos sin problemas de dependencias. En conjunto, el uso de contenedores mejora la organización, portabilidad y eficiencia en el desarrollo de software.